

Fan Director Studio — GitHub Actions CI/CD Pipeline

Complete CI/CD workflows for automated testing, building, and deployment across frontend, backend, and database.

Workflow Files Structure

```
.github/workflows/  
├─ frontend-ci.yml          # Frontend: lint, test, build  
├─ backend-ci.yml          # Backend: lint, test, API validation  
├─ database-migrations.yml # Database: migration testing  
├─ deploy-staging.yml      # Deploy to staging environment  
├─ deploy-production.yml   # Deploy to production  
└─ security-audit.yml      # Security scanning & dependency audit
```

1. Frontend CI Pipeline

File: `.github/workflows/frontend-ci.yml`

```
name: Frontend CI  
  
on:  
  push:  
    branches: [main, develop]  
    paths:  
      - 'frontend/**'  
      - '.github/workflows/frontend-ci.yml'  
  pull_request:  
    branches: [main, develop]  
    paths:  
      - 'frontend/**'  
  
jobs:  
  test:  
    runs-on: ubuntu-latest  
  
    strategy:  
      matrix:  
        node-version: [18.x, 20.x]  
  
    steps:  
      - uses: actions/checkout@v4
```

```

- name: Setup Node.js ${ matrix.node-version }
  uses: actions/setup-node@v4
  with:
    node-version: ${ matrix.node-version }
    cache: 'npm'
    cache-dependency-path: 'frontend/package-lock.json'

- name: Install dependencies
  run: |
    cd frontend
    npm ci

- name: Lint code
  run: |
    cd frontend
    npm run lint

- name: Run unit tests
  run: |
    cd frontend
    npm run test:unit -- --coverage

- name: Run E2E tests
  run: |
    cd frontend
    npm run test:e2e

- name: Build for production
  run: |
    cd frontend
    npm run build

- name: Upload coverage reports
  uses: codecov/codecov-action@v3
  with:
    files: ./frontend/coverage/coverage-final.json
    flags: frontend
    name: frontend-coverage

- name: Upload build artifacts
  uses: actions/upload-artifact@v3
  if: success()
  with:
    name: frontend-build-${ matrix.node-version }
    path: frontend/dist
    retention-days: 30

```

```

lighthouse:
  runs-on: ubuntu-latest
  needs: test
  if: github.event_name == 'pull_request'

```

```

steps:
  - uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: '20.x'
      cache: 'npm'
      cache-dependency-path: 'frontend/package-lock.json'

  - name: Install dependencies
    run: |
      cd frontend
      npm ci

  - name: Build
    run: |
      cd frontend
      npm run build

  - name: Run Lighthouse CI
    uses: treosh/lighthouse-ci-action@v10
    with:
      configPath: './frontend/lighthouseci.json'
      uploadArtifacts: true
      temporaryPublicStorage: true

```

Package.json scripts:

```

{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint src --max-warnings 0",
    "test:unit": "vitest",
    "test:e2e": "cypress run",
    "test:watch": "vitest --watch",
    "format": "prettier --write src"
  }
}

```

2. Backend CI Pipeline

File: `.github/workflows/backend-ci.yml`

```

name: Backend CI

on:

```

```

push:
  branches: [main, develop]
  paths:
    - 'backend/**'
    - '.github/workflows/backend-ci.yml'
pull_request:
  branches: [main, develop]
  paths:
    - 'backend/**'

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_DB: test_db
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_password
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20.x'
          cache: 'npm'
          cache-dependency-path: 'backend/package-lock.json'

      - name: Install dependencies
        run: |
          cd backend
          npm ci

      - name: Lint code
        run: |
          cd backend
          npm run lint

      - name: Check TypeScript
        run: |
          cd backend
          npm run type-check

```

```

- name: Run unit tests
  env:
    DATABASE_URL: postgres://test_user:test_password@localhost:5432/test_db
  run: |
    cd backend
    npm run test:unit -- --coverage

- name: Run integration tests
  env:
    DATABASE_URL: postgres://test_user:test_password@localhost:5432/test_db
  run: |
    cd backend
    npm run test:integration

- name: Validate OpenAPI spec
  run: |
    cd backend
    npm run validate:api

- name: Upload coverage reports
  uses: codecov/codecov-action@v3
  with:
    files: ./backend/coverage/coverage-final.json
    flags: backend
    name: backend-coverage

```

security:

runs-on: ubuntu-latest

steps:

```

- uses: actions/checkout@v4

- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20.x'

- name: Run npm audit
  run: |
    cd backend
    npm audit --audit-level=moderate

- name: Run OWASP Dependency Check
  uses: dependency-check/Dependency-Check_Action@main
  with:
    path: './backend'
    format: 'JSON'
    args: >-
      --enable-experimental

```

Package.json scripts:

```
{
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js",
    "lint": "eslint src --max-warnings 0",
    "type-check": "tsc --noEmit",
    "test:unit": "jest",
    "test:integration": "jest --testPathPattern=integration",
    "validate:api": "swagger-cli validate src/openapi.yaml"
  }
}
```

3. Database Migrations Pipeline

File: `.github/workflows/database-migrations.yml`

```
name: Database Migrations

on:
  push:
    branches: [main, develop]
    paths:
      - 'db/migrations/**'
      - '.github/workflows/database-migrations.yml'
  pull_request:
    branches: [main, develop]
    paths:
      - 'db/migrations/**'

jobs:
  validate-migrations:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_DB: migration_test
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_password
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432
```

```

steps:
  - uses: actions/checkout@v4

  - name: Install Flyway CLI
    run: |
      wget -qO flyway.tar.gz https://repo1.maven.org/maven2/org/flywaydb/flyway-commandline/9.22.3/flyway-commandline-9.22.3.tar.gz
      tar -xzf flyway.tar.gz
      export PATH="$PWD/flyway-9.22.3:$PATH"

  - name: Validate migration syntax
    run: |
      cd db
      find migrations -name "*.sql" -exec sqlparse --validate {} \;

  - name: Test migrations forward
    env:
      FLYWAY_URL: jdbc:postgresql://localhost:5432/migration_test
      FLYWAY_USER: test_user
      FLYWAY_PASSWORD: test_password
      FLYWAY_LOCATIONS: filesystem:./migrations
    run: |
      cd db
      flyway migrate

  - name: Test migrations rollback (if supported)
    env:
      FLYWAY_URL: jdbc:postgresql://localhost:5432/migration_test
      FLYWAY_USER: test_user
      FLYWAY_PASSWORD: test_password
      FLYWAY_LOCATIONS: filesystem:./migrations
    run: |
      cd db
      # Verify schema is correctly created
      psql -h localhost -U test_user -d migration_test -c "\dt"

  - name: Generate migration report
    run: |
      cd db
      echo "# Migration Validation Report" > migration_report.md
      echo "✅ All migrations validated successfully" >> migration_report.md

  - name: Comment PR with migration status
    if: github.event_name == 'pull_request'
    uses: actions/github-script@v7
    with:
      script: |
        github.rest.issues.createComment({
          issue_number: context.issue.number,
          owner: context.repo.owner,
          repo: context.repo.repo,
          body: '✅ Database migrations validated successfully'
        })

```

4. Deploy to Staging

File: `.github/workflows/deploy-staging.yml`

```
name: Deploy to Staging

on:
  push:
    branches: [develop]
  workflow_dispatch:
  inputs:
    environment:
      description: 'Environment to deploy to'
      required: true
      default: 'staging'

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.fandirectorstudio.com

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20.x'
          cache: 'npm'

      - name: Build frontend
        run: |
          cd frontend
          npm ci
          npm run build
        env:
          VITE_API_BASE_URL: https://api-staging.fandirectorstudio.com/v1
          VITE_ENVIRONMENT: staging

      - name: Build backend
        run: |
          cd backend
          npm ci
          npm run build

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
```

```

aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
aws-region: us-east-1

- name: Deploy frontend to S3
  run: |
    aws s3 sync frontend/dist s3://fan-director-staging-frontend \
      --delete \
      --cache-control "max-age=31536000,immutable" \
      --exclude "index.html" && \
    aws s3 cp frontend/dist/index.html s3://fan-director-staging-frontend/index.html \
      --cache-control "max-age=0,no-cache,no-store,must-revalidate"

- name: Invalidate CloudFront cache
  run: |
    aws cloudfront create-invalidation \
      --distribution-id ${ secrets.STAGING_CLOUDFRONT_DIST_ID } \
      --paths "/*"

- name: Push Docker image to ECR
  run: |
    aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.us-east-1.amazonaws.com
    docker build -t fan-director-api:staging -f backend/Dockerfile.staging backend/
    docker tag fan-director-api:staging ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.us-east-1.amazonaws.com/fan-director-api:staging
    docker push ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.us-east-1.amazonaws.com/fan-director-api:staging

- name: Deploy to ECS
  run: |
    aws ecs update-service \
      --cluster staging-cluster \
      --service fan-director-api \
      --force-new-deployment

- name: Run smoke tests
  run: |
    npm install -g newman
    newman run postman_collection.json \
      -e environments/staging.json \
      --reporters cli,json \
      --reporter-json-export smoke-test-results.json

- name: Notify Slack
  if: always()
  uses: slackapi/slack-github-action@v1.24.0
  with:
    webhook-url: ${ secrets.SLACK_WEBHOOK_STAGING }
    payload: |
      {
        "text": "Staging deployment ${ job.status }",
        "blocks": [
          {
            "type": "section",
            "text": {

```

```

        "type": "mrkdwn",
        "text": "**Staging Deployment* ${ job.status }}\n*Commit:* <${ github.server_url }/${ github.ref }}*  

    }  

  }  

]  

}

```

5. Deploy to Production

File: `.github/workflows/deploy-production.yml`

```

name: Deploy to Production

on:
  push:
    branches: [main]
    tags:
      - 'v*'
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://fandirectorstudio.com

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20.x'
          cache: 'npm'

      - name: Create GitHub Release
        uses: actions/create-release@v1
        if: startsWith(github.ref, 'refs/tags/')
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        with:
          tag_name: ${ github.ref }
          release_name: Release ${ github.ref }
          body: |
            Production release

            Commit: ${ github.sha }
            Author: ${ github.actor }

```

```

    draft: false
    prerelease: false

- name: Build frontend
  run: |
    cd frontend
    npm ci
    npm run build
  env:
    VITE_API_BASE_URL: https://api.fandirectorstudio.com/v1
    VITE_ENVIRONMENT: production

- name: Build backend
  run: |
    cd backend
    npm ci
    npm run build

- name: Run database migrations
  env:
    DATABASE_URL: ${ secrets.PROD_DATABASE_URL }
  run: |
    cd db
    npm install -g flyway-cli
    flyway migrate

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${ secrets.AWS_ROLE_TO_ASSUME }
    aws-region: us-east-1

- name: Deploy frontend to S3 + CloudFront
  run: |
    aws s3 sync frontend/dist s3://fan-director-prod-frontend \
      --delete \
      --cache-control "max-age=31536000,immutable" \
      --exclude "index.html"
    aws s3 cp frontend/dist/index.html s3://fan-director-prod-frontend/index.html \
      --cache-control "max-age=0,no-cache,no-store,must-revalidate"
    aws cloudfront create-invalidation \
      --distribution-id ${ secrets.PROD_CLOUDFRONT_DIST_ID } \
      --paths "/*"

- name: Push Docker image to ECR
  run: |
    aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.us-east-1.amazonaws.com
    docker build -t fan-director-api:${ secrets.GITHUB_SHA } -f backend/Dockerfile backend/
    docker tag fan-director-api:${ secrets.GITHUB_SHA } ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.us-east-1.amazonaws.com
    docker tag fan-director-api:${ secrets.GITHUB_SHA } ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.us-east-1.amazonaws.com
    docker push ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.us-east-1.amazonaws.com/fan-director-api:${ secrets.GITHUB_SHA }
    docker push ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.us-east-1.amazonaws.com/fan-director-api:latest

```

```

- name: Update ECS service
  run: |
    aws ecs update-service \
      --cluster production-cluster \
      --service fan-director-api \
      --force-new-deployment

- name: Wait for ECS deployment
  run: |
    aws ecs wait services-stable \
      --cluster production-cluster \
      --services fan-director-api

- name: Run production smoke tests
  run: |
    npm install -g newman
    newman run postman_collection.json \
      -e environments/production.json \
      --reporters cli,json \
      --reporter-json-export smoke-test-results.json

- name: Enable CloudFront WAF rules
  run: |
    aws wafv2 update-ip-set \
      --name fan-director-prod-allowed-ips \
      --scope CLOUDFRONT \
      --region us-east-1 \
      --id ${ secrets.WAF_IP_SET_ID } \
      --addresses ["0.0.0.0/0"]

- name: Notify stakeholders
  if: always()
  uses: slackapi/slack-github-action@v1.24.0
  with:
    webhook-url: ${ secrets.SLACK_WEBHOOK_PROD }
    payload: |
      {
        "text": "Production deployment ${ job.status }",
        "blocks": [
          {
            "type": "section",
            "text": {
              "type": "mrkdwn",
              "text": "*🚀 Production Deployment* ${ job.status }\n*Release:* ${ github.ref }\n*Commit:* <
            }
          }
        ]
      }

- name: Create Sentry release
  run: |
    npm install -g @sentry/cli
    sentry-cli releases create ${ github.ref }

```

```

sentry-cli releases files ${GITHUB_REF} upload-sourcemaps ./frontend/dist
sentry-cli releases finalize ${GITHUB_REF}
env:
  SENTRY_AUTH_TOKEN: ${SECRETS.SENTRY_AUTH_TOKEN}
  SENTRY_ORG: fan-director
  SENTRY_PROJECT: frontend

```

6. Security Audit Pipeline

File: `.github/workflows/security-audit.yml`

```

name: Security Audit

on:
  push:
    branches: [main, develop]
  schedule:
    # Run every Monday at 9am UTC
    - cron: '0 9 * * 1'
  workflow_dispatch:

jobs:
  security-scan:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Run npm audit
        run: npm audit --audit-level=high || true

      - name: Run Snyk security scan
        uses: snyk/actions/node@master
        env:
          SNYK_TOKEN: ${SECRETS.SNYK_TOKEN}
        with:
          args: --severity-threshold=high --all-projects

      - name: Run OWASP Dependency Check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          path: '.'
          format: 'JSON'
          args: >-
            --enable-experimental
            --project "Fan Director Studio"

      - name: Upload OWASP report
        uses: actions/upload-artifact@v3

```

```

    if: always()
    with:
      name: dependency-check-report
      path: reports

- name: Run SonarQube scan
  uses: SonarSource/sonarcloud-github-action@master
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  with:
    args: >-
      -Dsonar.projectKey=fan-director-studio
      -Dsonar.organization=fan-director

container-scan:
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v4

    - name: Build Docker image
      run: docker build -f backend/Dockerfile -t fan-director-api:scan backend/

    - name: Run Trivy container scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: 'fan-director-api:scan'
        format: 'sarif'
        output: 'trivy-results.sarif'

    - name: Upload Trivy results to GitHub Security
      uses: github/codeql-action/upload-sarif@v2
      if: always()
      with:
        sarif_file: 'trivy-results.sarif'

code-quality:
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '20.x'
        cache: 'npm'

    - name: Install dependencies
      run: |

```

```

    npm ci --workspaces

- name: Run ESLint
  run: npm run lint --workspaces || true

- name: Run CodeQL analysis
  uses: github/codeql-action/init@v2
  with:
    languages: javascript

- name: Perform CodeQL analysis
  uses: github/codeql-action/analyze@v2

- name: Create security report
  if: always()
  run: |
    echo "# Security Audit Report" > security-report.md
    echo "- npm audit: Complete" >> security-report.md
    echo "- Snyk: Complete" >> security-report.md
    echo "- OWASP Dependency Check: Complete" >> security-report.md
    echo "- Container Scan: Complete" >> security-report.md
    echo "- CodeQL: Complete" >> security-report.md

- name: Comment PR with security status
  if: github.event_name == 'pull_request'
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: '✅ Security audit completed successfully'
      })

```

Setup Instructions

1. Create Workflow Files

```

# Create .github/workflows directory
mkdir -p .github/workflows

# Copy all workflow files from above
# Each YAML file goes in .github/workflows/

```

2. Configure Secrets

Go to **GitHub Settings** → **Secrets and variables** → **Actions** and add:

```

# AWS
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_ACCOUNT_ID
AWS_ROLE_TO_ASSUME

# Staging
STAGING_CLOUDFRONT_DIST_ID

# Production
PROD_CLOUDFRONT_DIST_ID
PROD_DATABASE_URL
WAF_IP_SET_ID

# Third-party services
SNYK_TOKEN
SONAR_TOKEN
SENTRY_AUTH_TOKEN

# Slack webhooks
SLACK_WEBHOOK_STAGING
SLACK_WEBHOOK_PROD

# GitHub
GITHUB_TOKEN (auto-provided)

```

3. Environment Configuration

Create `environments/staging.json` and `environments/production.json`:

```

{
  "name": "Staging",
  "values": [
    {
      "key": "base_url",
      "value": "https://api-staging.fandirectorstudio.com/v1",
      "enabled": true
    },
    {
      "key": "authToken",
      "value": "",
      "enabled": true
    }
  ]
}

```

4. Enable Branch Protection

Go to **Settings** → **Branches** → **Add rule for** `main` and `develop`:

- ✔ Require a pull request before merging
- ✔ Require status checks to pass before merging
- ✔ Require branches to be up to date before merging
- ✔ Require code reviews before merging (at least 1)
- ✔ Require administrators to follow the same rules

Workflow Triggers

Workflow	Trigger	When
frontend-ci.yml	Push/PR to <code>main</code> , <code>develop</code> (frontend paths)	On every commit
backend-ci.yml	Push/PR to <code>main</code> , <code>develop</code> (backend paths)	On every commit
database-migrations.yml	Push/PR (db/migrations)	On migration changes
deploy-staging.yml	Push to <code>develop</code>	On every merge to develop
deploy-production.yml	Push to <code>main</code> or tag push	On release/production push
security-audit.yml	Weekly (Monday 9am UTC) + manual	Regular security checks

Environment Variables

Frontend (.env)

```
VITE_API_BASE_URL=https://api.fandirectorstudio.com/v1
VITE_ENVIRONMENT=production
VITE_SENTRY_DSN=https://key@sentry.io/project
```

Backend (.env)

```
DATABASE_URL=postgresql://user:password@host/db
REDIS_URL=redis://localhost:6379
JWT_SECRET=your-secret-key
ENVIRONMENT=production
LOG_LEVEL=info
```

Monitoring & Alerts

GitHub Status Dashboard

<https://github.com/your-org/fan-director-studio/actions>

Slack Integration

Workflows automatically notify:

- `#deployments` (staging/prod deploys)
- `#security` (security audit results)
- `#development` (CI failures)

Status Badges

Add to README.md:

```



```

Rollback Procedure

Quick Rollback (Production)

```
# Revert to previous Docker image
aws ecs update-service \
  --cluster production-cluster \
  --service fan-director-api \
  --force-new-deployment \
  --region us-east-1

# Or redeploy a specific commit
git push origin <previous-commit-sha>
```

Database Rollback

```
# Flyway undo (manual - no automatic rollback)
cd db
flyway undo
```

Best Practices

1. **Use branch protection** on `main` and `develop`
 2. **Require at least 1 code review** before merging
 3. **Keep secrets in GitHub Secrets**, never in code
 4. **Run security checks weekly** in addition to on-push
 5. **Monitor deployment logs** in CloudWatch
 6. **Keep workflow files DRY** using reusable workflows (if needed)
 7. **Test rollback procedures** regularly
 8. **Pin action versions** (avoid `@master`, use `@v4`)
-

Troubleshooting

Common Issues

Workflow not triggering

- Check branch name (main vs master)
- Verify paths filter (if using)
- Check GitHub Actions is enabled

Tests failing in CI but passing locally

- Different Node versions? Check matrix
- Missing environment variables? Add to Secrets
- Database connection? Check service container

Deployment stuck

- Check ECS service status
- Review CloudFormation events
- Check IAM permissions

Security scan failing

- Update dependencies: `npm update`
- Review SARIF report for vulnerabilities
- Use `npm audit fix` for auto-fixes